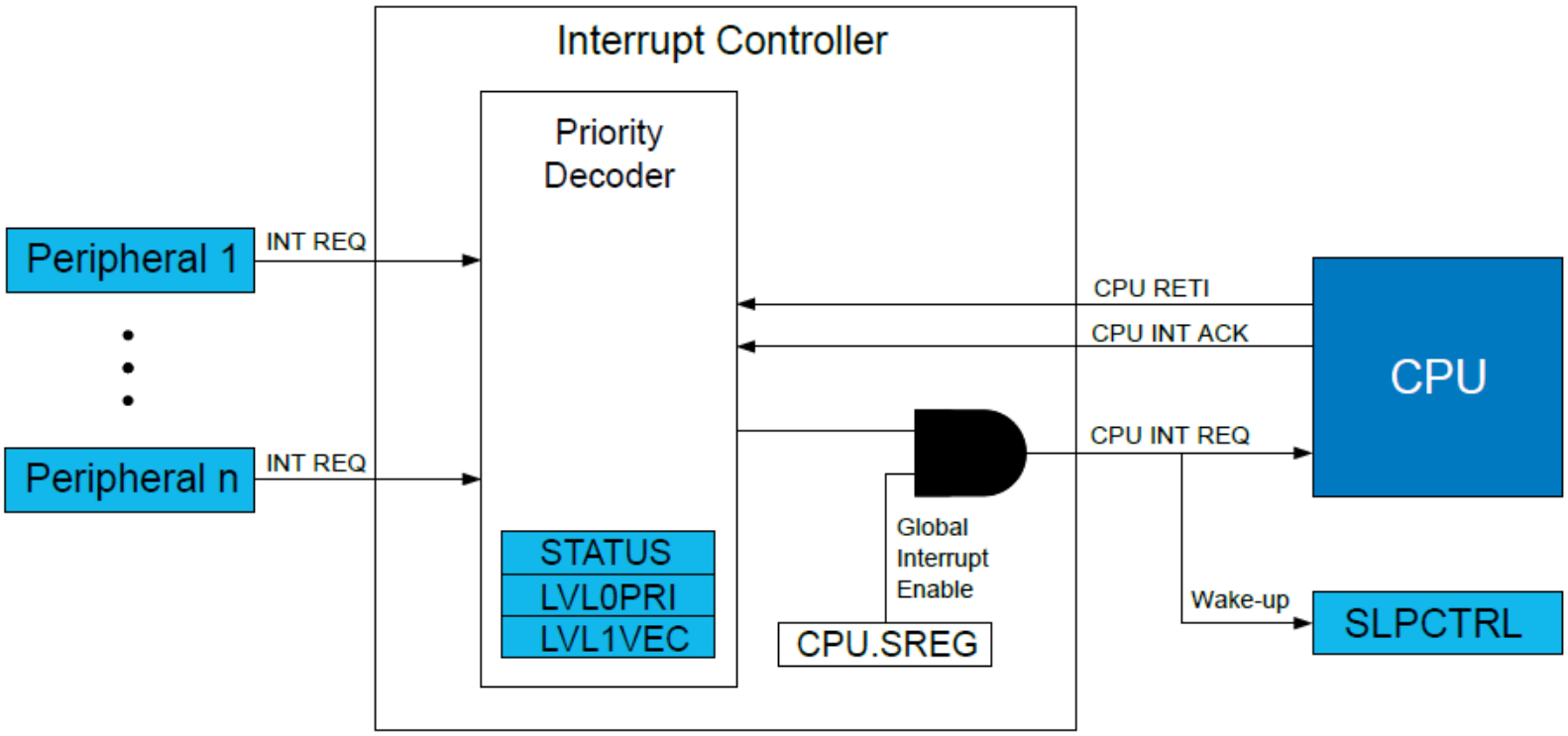
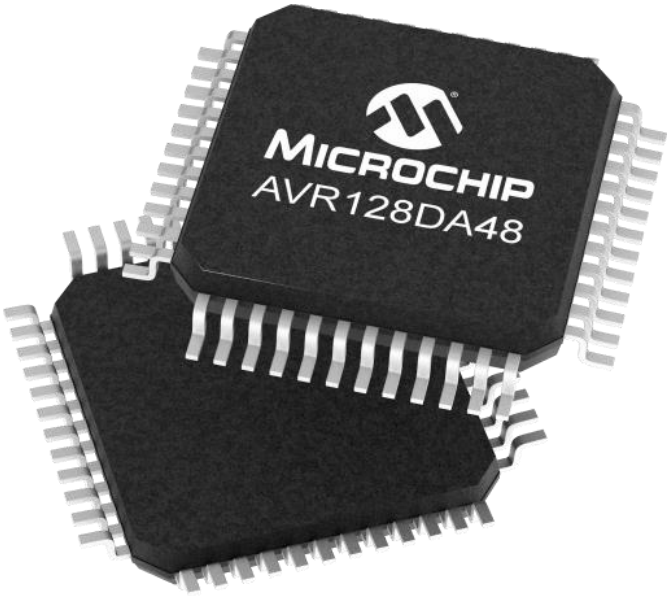
“Bare-Metal Embedded Systems with AVR128DA48 Course”

Day 23 - Interrupt Architecture





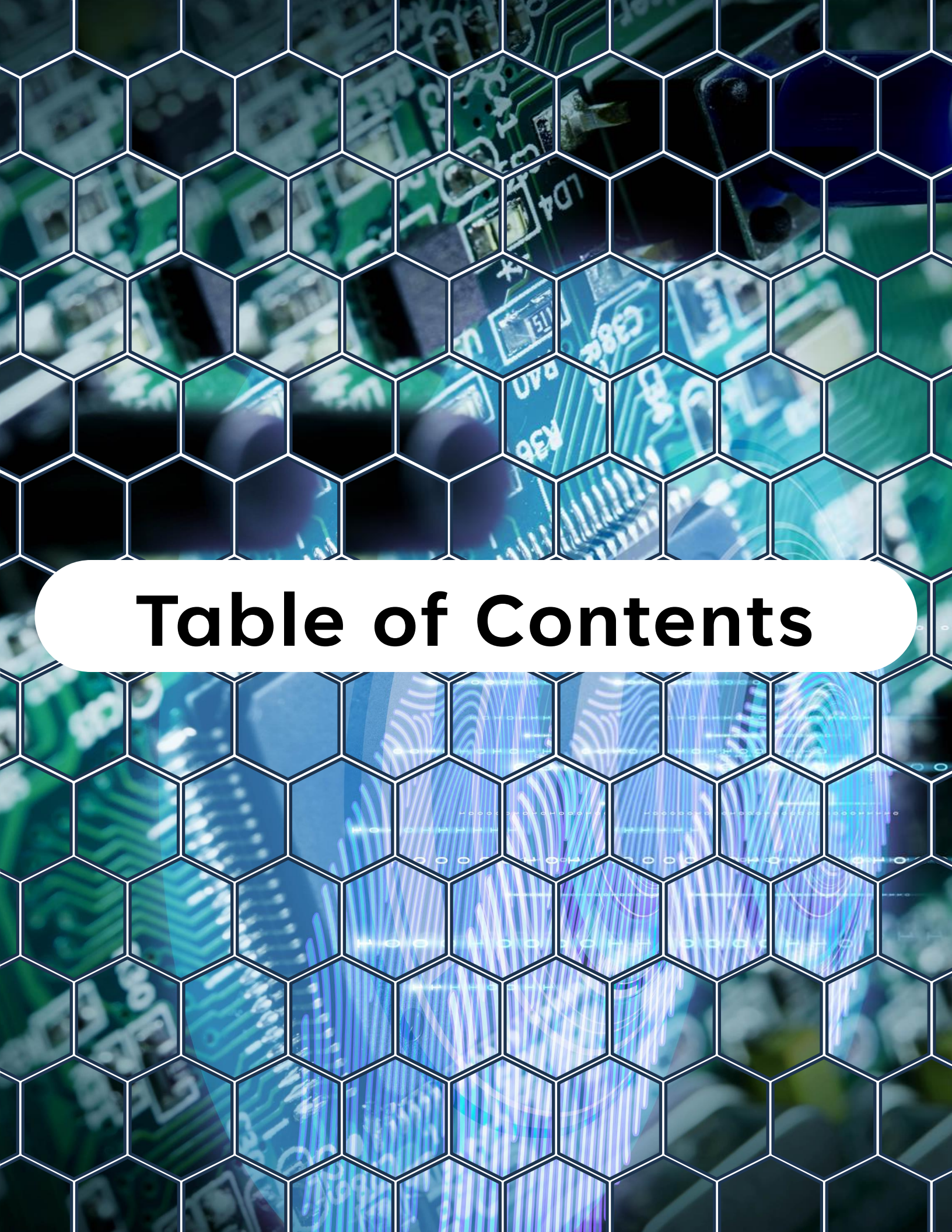
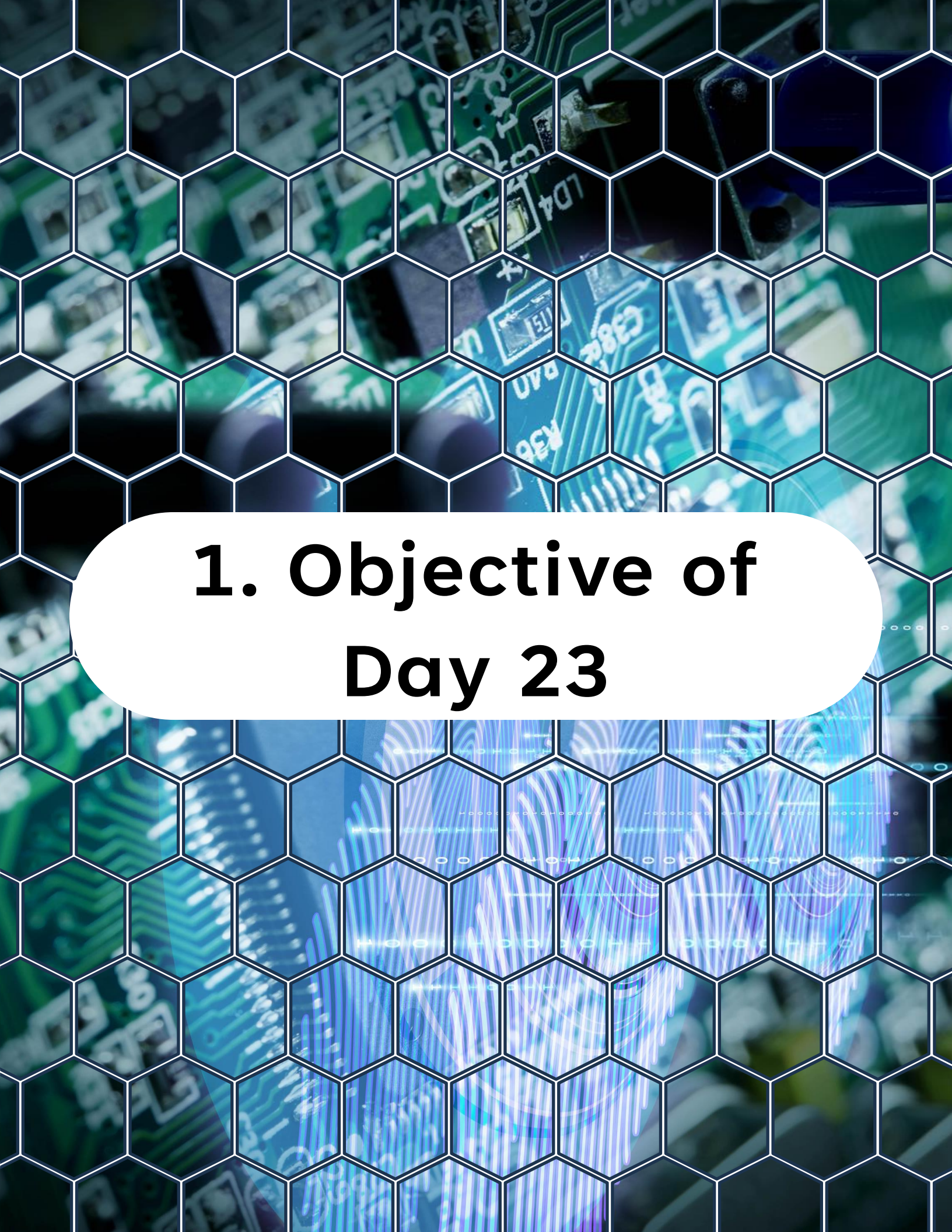


Table of Contents

Table of Contents



1. Objective of Day 23

1. Objective of Day 23

So far, our programs have mostly used polling or hardware pipelines.

Today we focus on one of the most fundamental concepts in embedded systems:
Interrupts.

Interrupts allow peripherals to signal the CPU when something important happens, without forcing the CPU to constantly check status flags.

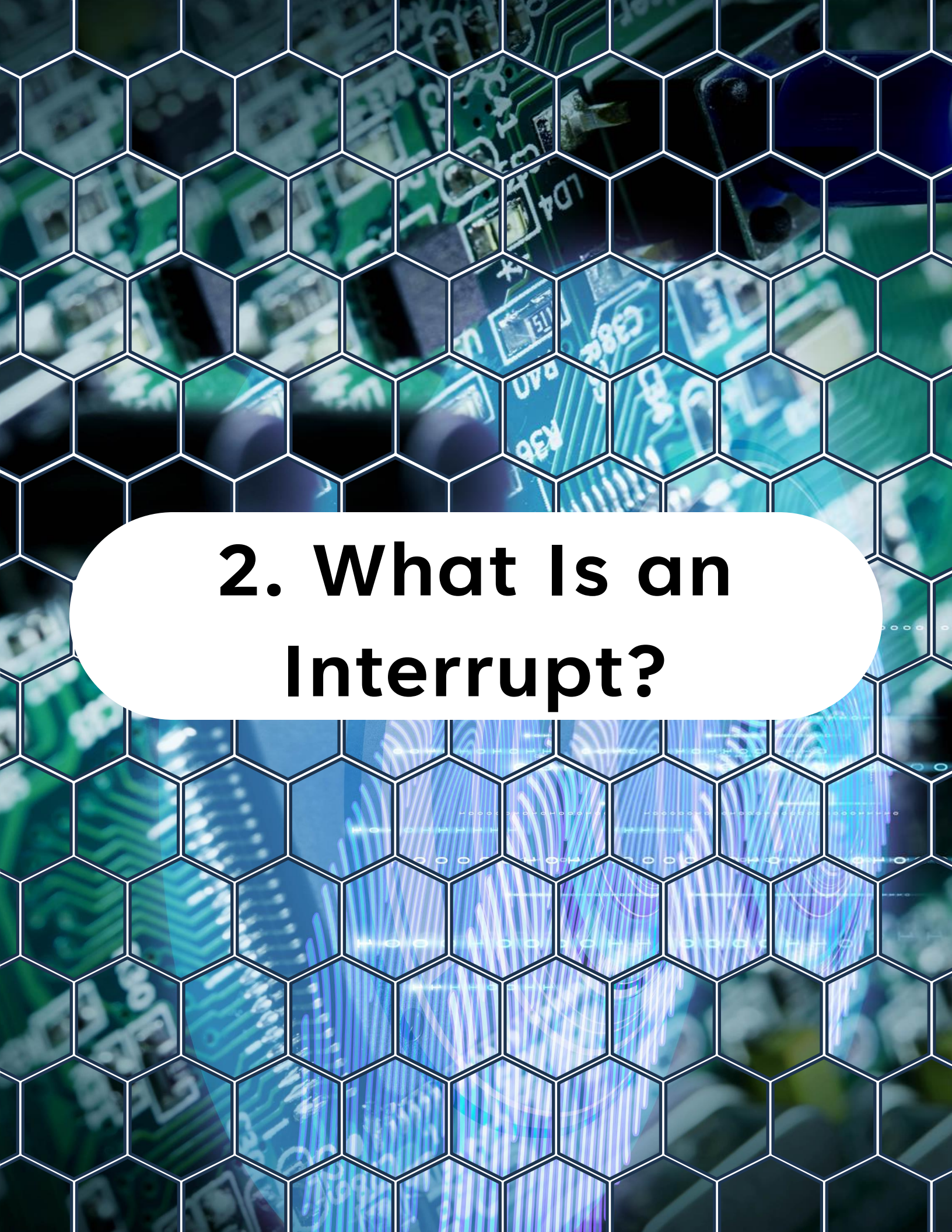
By the end of today, you will understand:

- What interrupts are and how they work
- How interrupt vectors are organized
- How interrupts are enabled and handled

1. Objective of Day 23

- Best practices for writing Interrupt Service Routines (ISRs)
- How to design systems with multiple interrupts safely

We will finish with a lab that builds a small multi-interrupt system.



2. What Is an Interrupt?

2. What Is an Interrupt?

An interrupt is a hardware signal that temporarily stops normal program execution so the CPU can respond to an event.

Typical interrupt sources:

- Timer overflow
- ADC conversion complete
- UART receive complete
- External pin change
- Comparator threshold crossing

Without interrupts, the CPU must continuously poll peripheral flags.

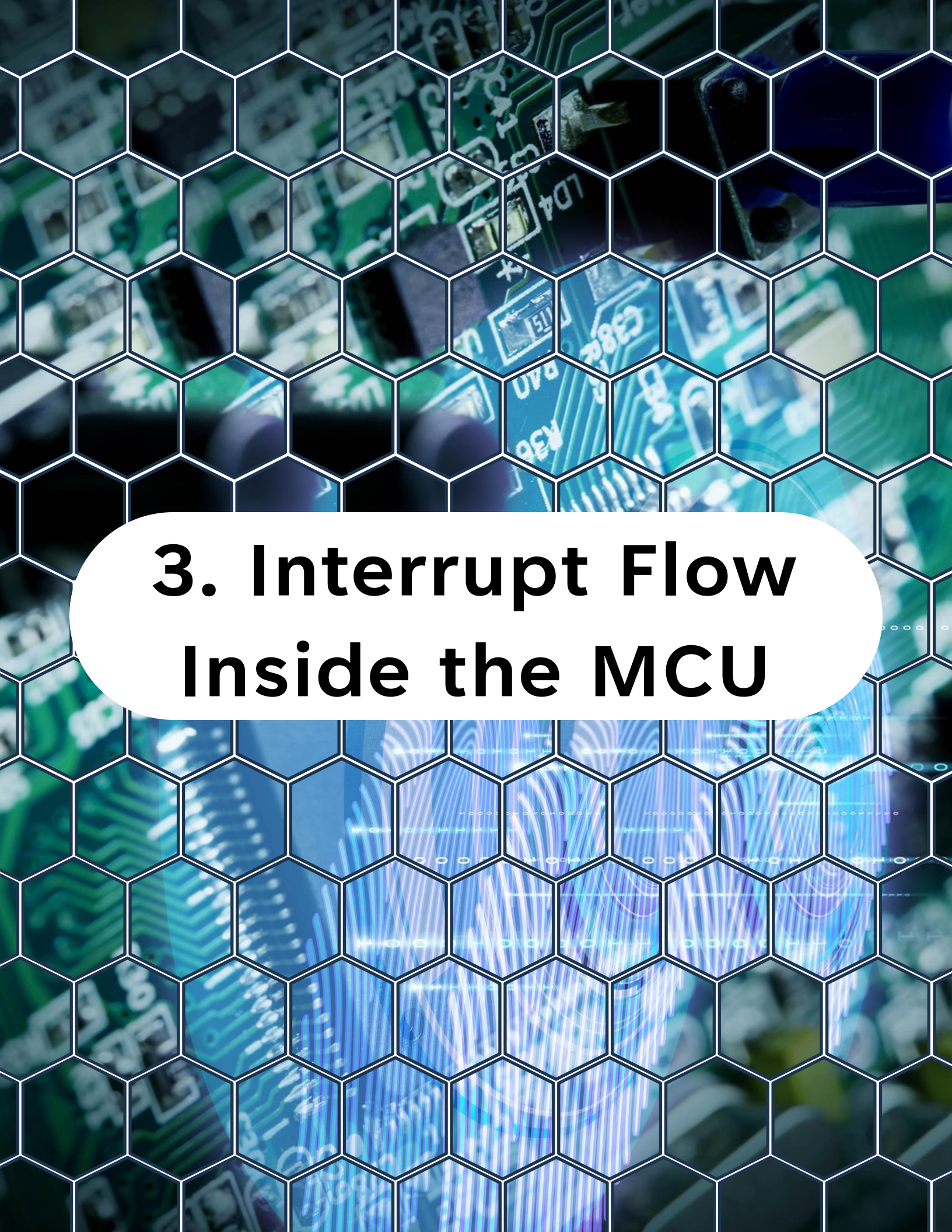
2. What Is an Interrupt?

Example polling loop:

```
// While the ADC is busy, wait for the
// result to be ready
while (!(ADC0.INTFLAGS & ADC_RESRDY_bm))
{
}
```

With interrupts:

The CPU continues running other code, and the peripheral notifies the CPU automatically.



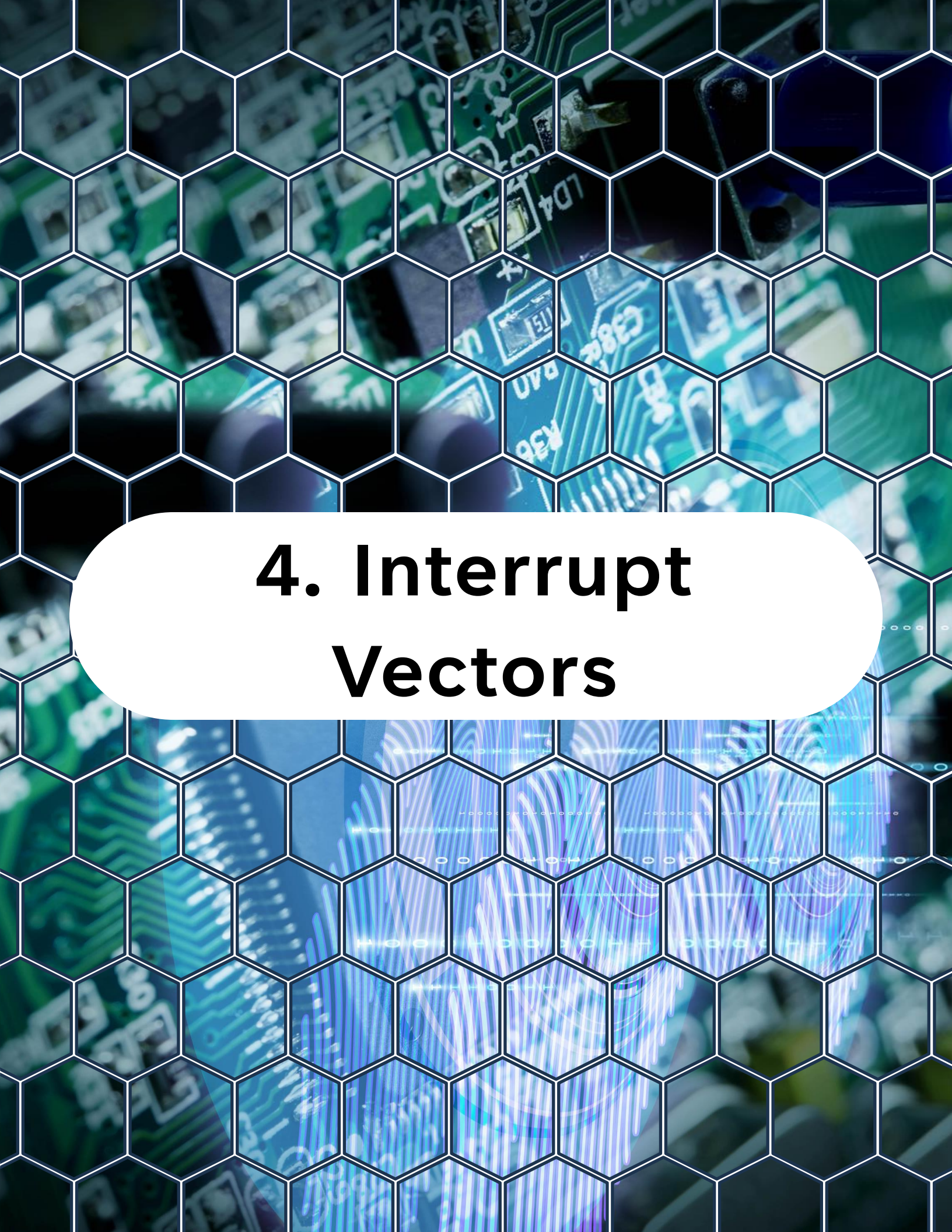
3. Interrupt Flow Inside the MCU

3. Interrupt Flow Inside the MCU

When an interrupt occurs, the following happens:

1. Peripheral sets interrupt flag
2. CPU finishes current instruction
3. Program counter is pushed to stack
4. CPU jumps to interrupt vector
5. ISR executes
6. RETI instruction returns to main program

This entire process typically takes only a few clock cycles.



4. Interrupt Vectors

4. Interrupt Vectors

Each interrupt source has a dedicated vector.

A vector is simply an address in memory where the ISR begins.

Example vectors include:

- TCA0 overflow
- TCB0 capture
- ADC result ready
- RTC overflow
- USART RX complete

The compiler links your ISR to the correct vector using the ISR() macro.

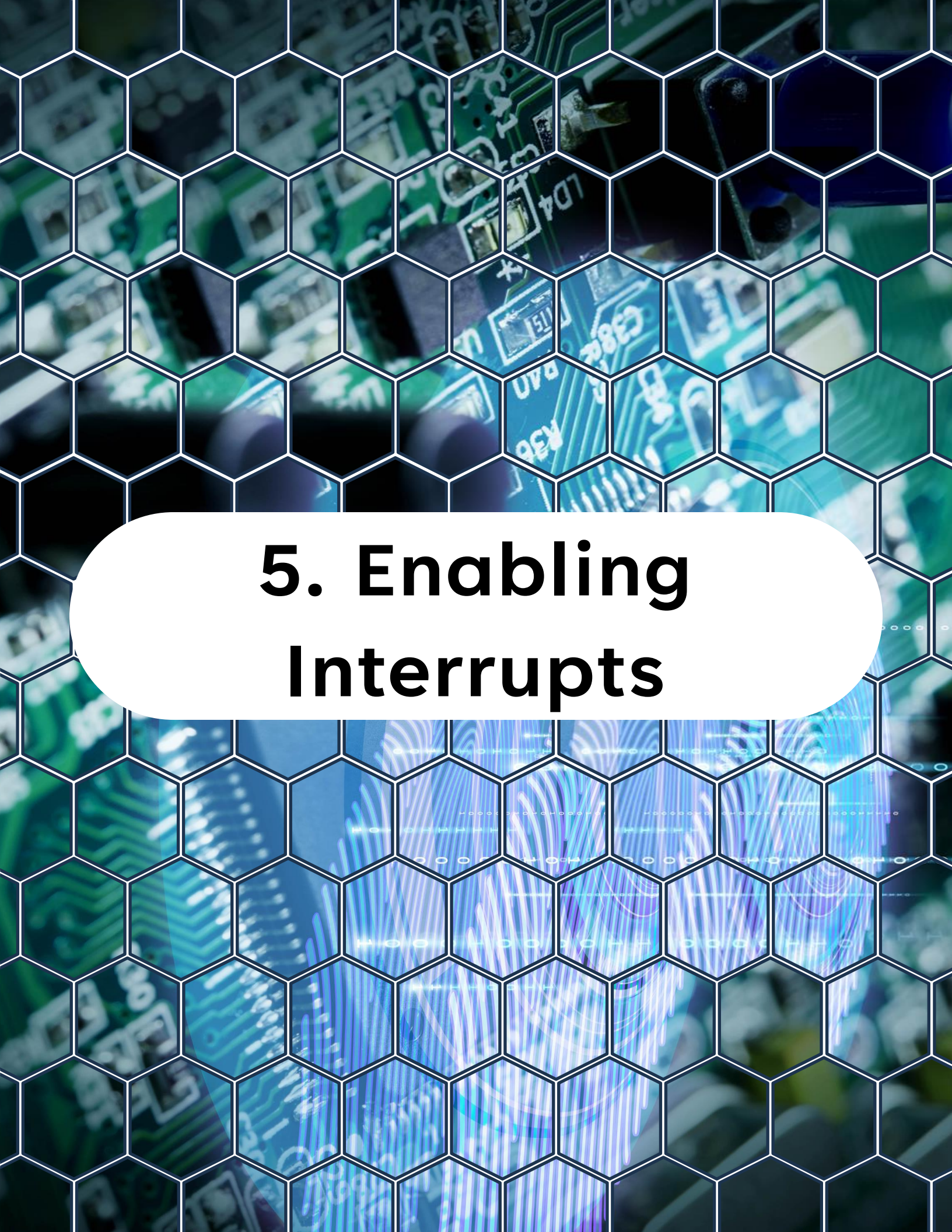
4. Interrupt Vectors

Examples:

```
// Interrupt handler for ADC0 result ready
ISR(ADC0_RESRDY_vect)
{
    /* Interrupt handler */
}

// Interrupt handler for TCA0 overflow
ISR(TCA0_OVF_vect)
{
    /* Interrupt handler */
}

// Interrupt handler for TCB0 interrupt
ISR(TCB0_INT_vect)
{
    /* Interrupt handler */
}
```



5. Enabling Interrupts

5. Enabling Interrupts

Interrupts require two things:

- Peripheral interrupt enable
- Global interrupt enable

5.1 Peripheral Interrupt Enable

Each peripheral has its own interrupt enable register.

Example for ADC:

```
1 // Enable ADC interrupt
2 ADC0.INTCTRL = ADC_RESRDY_bm;
```

Example for TCB:

```
1 // Enabling the TCB0 interrupt
2 TCB0.INTCTRL = TCB_CAPT_bm;
```

5. Enabling Interrupts

5.2 Global Interrupt Enable

Even if a peripheral enables its interrupt, the CPU will ignore it until global interrupts are enabled.

This is done using:

```
sei();
```

To disable interrupts:

```
cli();
```

These macros are provided by

```
<avr/interrupt.h>
```

6. Writing an Interrupt Service Routine (ISR)

6. Writing an Interrupt Service Routine (ISR)

An ISR must follow specific rules.

Basic syntax:

```
ISR(vector_name)
{
    /* Interrupt code */
}
```

Example:

```
// Interrupt service routine for ADC result ready
ISR(ADC0_RESRDY_vect)
{
    // Clear the interrupt flag
    ADC0.INTFLAGS = ADC_RESRDY_bm;

    // Read the ADC result
    uint16_t result = ADC0.RES;
}
```

The ISR name must match the vector name defined in the device header file.

7. ISR Best Practices

7. ISR Best Practices

Poor ISR design can cause serious problems.

Follow these rules:

7.1 Keep ISRs Short

An ISR should execute as quickly as possible.

Bad example:

```
ISR(...)
{
    printf("ADC value = %d", value);
}
```

Good example:

```
ISR(...)
{
    adc_value = ADC0.RES;
}
```

Process the data later in the main loop.

7. ISR Best Practices

7.2 Avoid Blocking Operations

Never use:

- Long delays
- Busy loops
- Slow communication

Inside an ISR.

7.3 Clear Interrupt Flags

Many peripherals require clearing the interrupt flag inside the ISR.

Example:

```
ADC0.INTFLAGS = ADC_RESRDY_bm;
```

If the flag is not cleared, the ISR may trigger again immediately.

7. ISR Best Practices

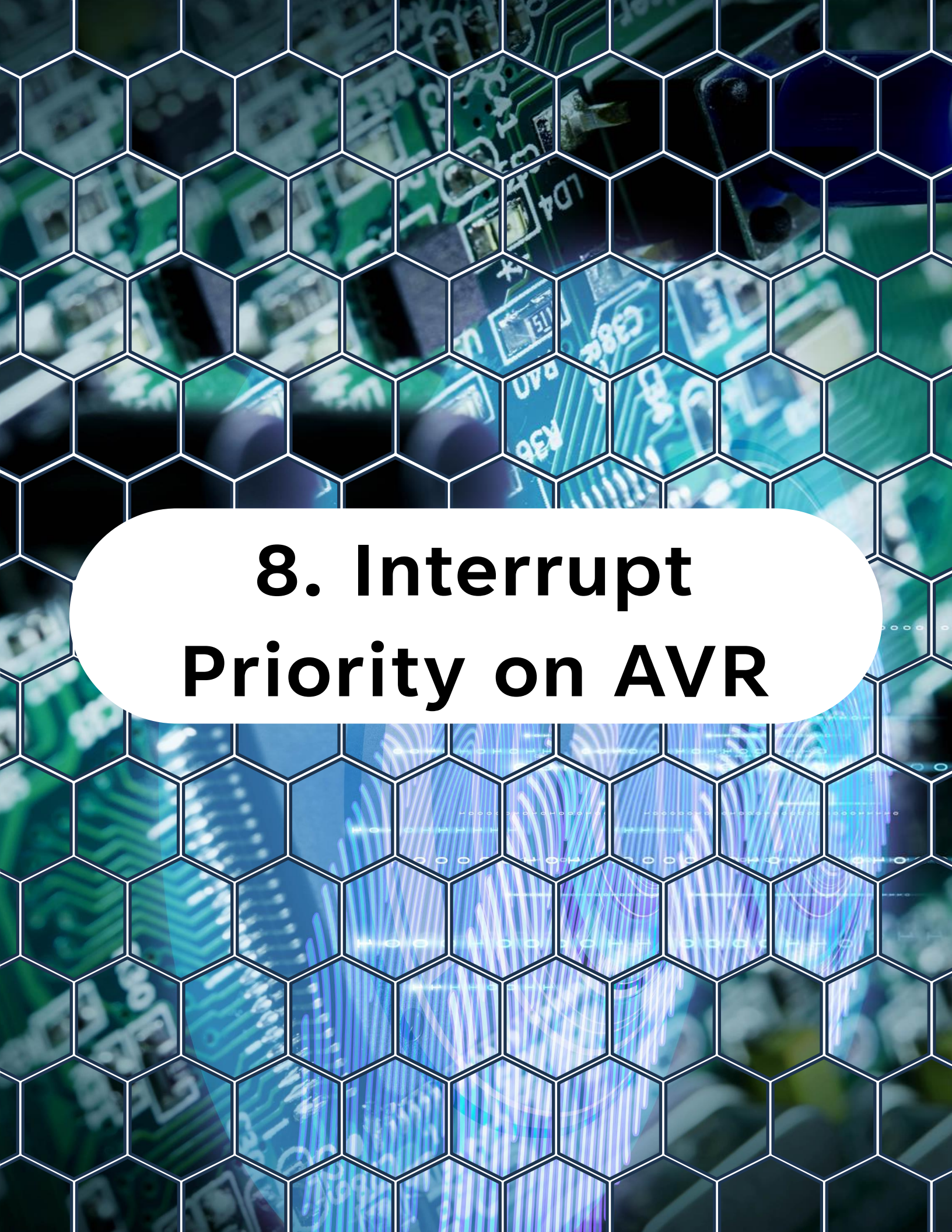
7.4 Use Volatile Variables

Variables shared between ISR and main code must be declared volatile.

Example:

```
volatile uint16_t adc_value;
```

This prevents the compiler from optimizing away memory accesses.



8. Interrupt Priority on AVR

8. Interrupt Priority on AVR

Unlike some architectures, the AVR core does not support dynamic interrupt priority levels.

Priority is determined by vector order in memory.

Lower address = higher priority.

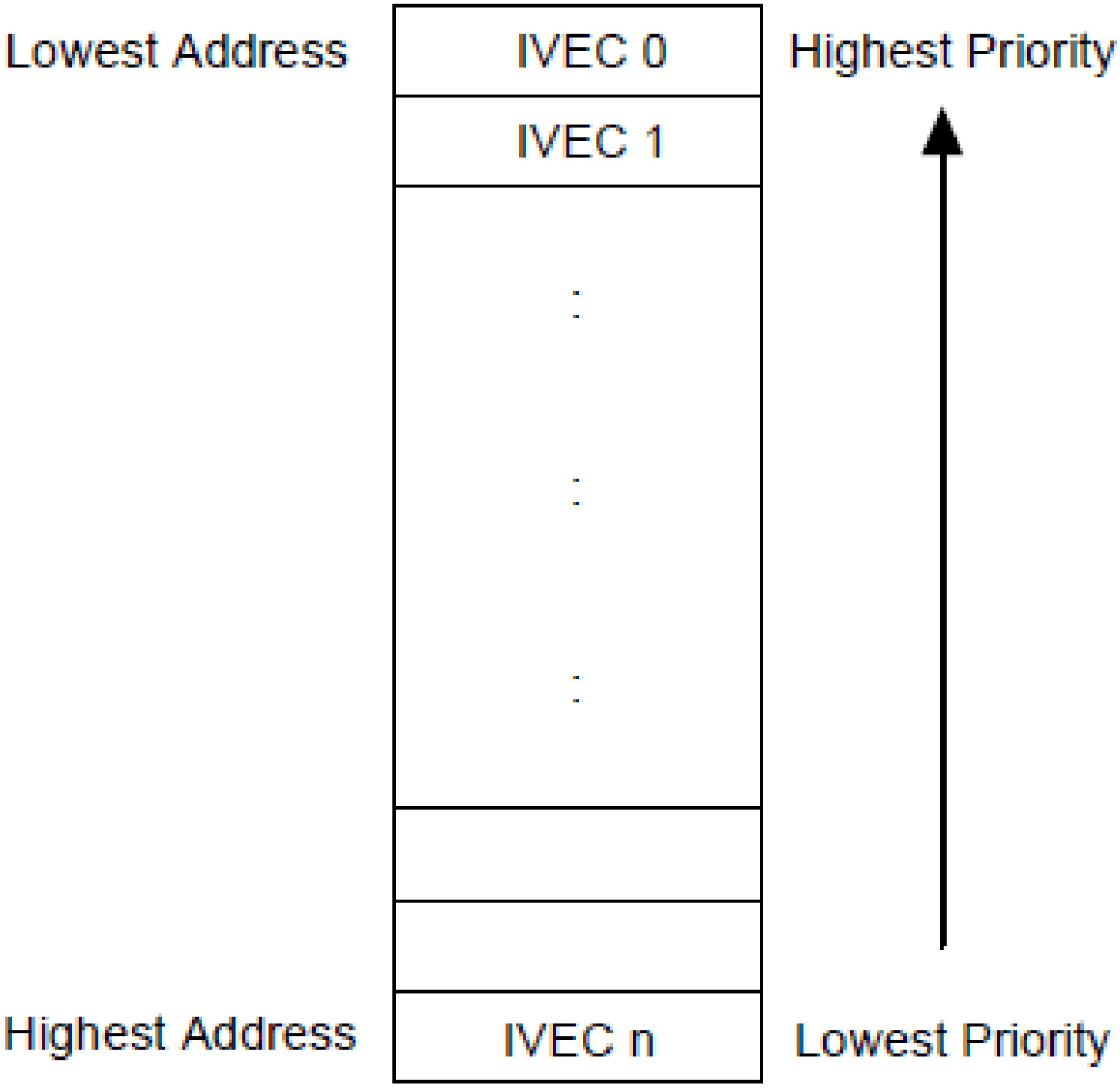
However, while an ISR is running:

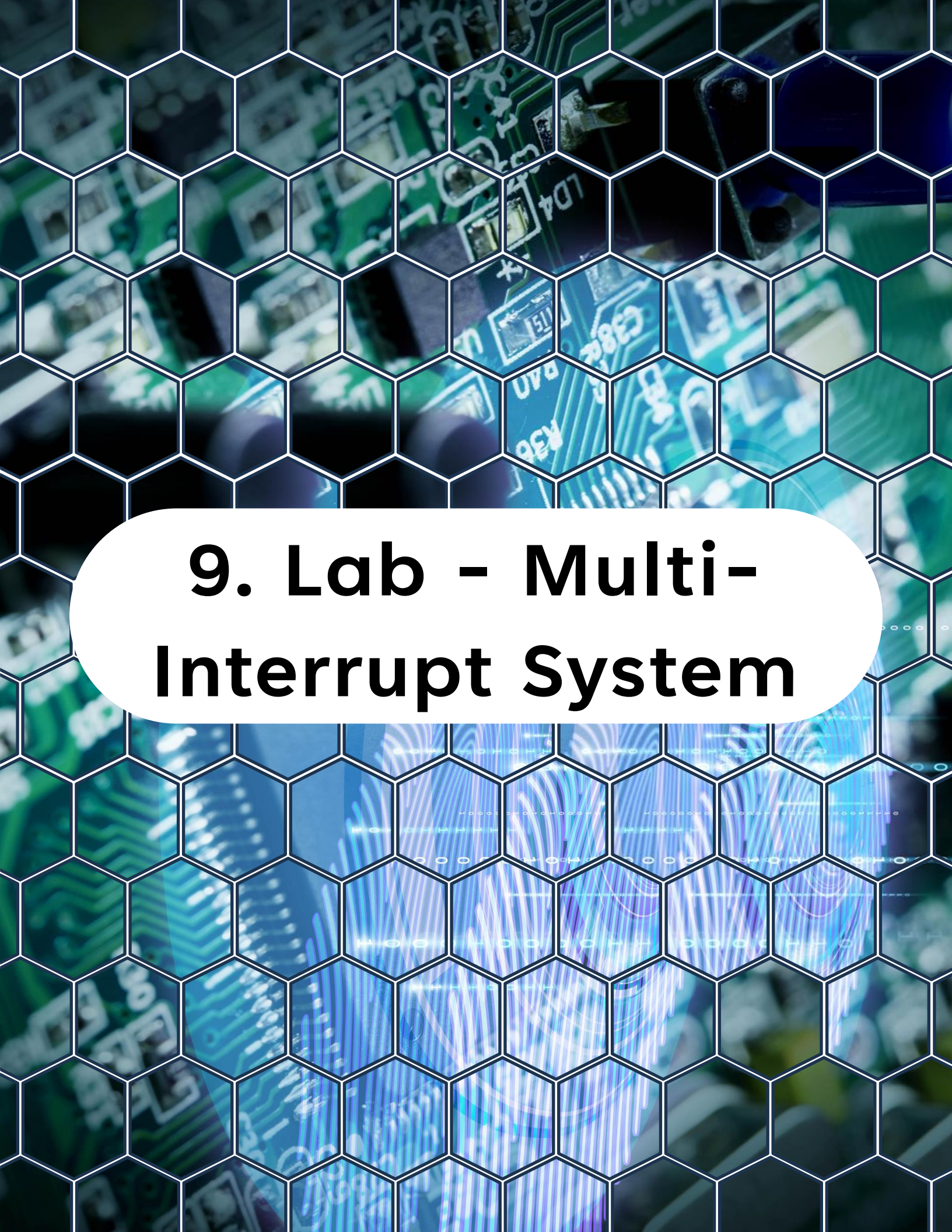
Other interrupts are automatically disabled.

This prevents nested interrupts unless explicitly re-enabled.

8. Interrupt Priority on AVR

Default Static Scheduling





9. Lab - Multi-Interrupt System

9. Lab - Multi-Interrupt System

Goal:

Create a system with two independent interrupts:

- RTC interrupt every 1 second
- ADC conversion interrupt when sampling completes

The main loop remains mostly idle.

9.1 LED Setup

```c
#define LED_PORT PORTC
// PC6 = LED0 on Curiosity Nano
#define LED_PIN  PIN6_bm

static void LED_init(void)
{
    // Set LED pin as output
    LED_PORT.DIRSET = LED_PIN;
}
```

9. Lab - Multi-Interrupt System

9.2 RTC Initialization

```
1  static void RTC_init(void)
2  {
3      // Configure RTC for 1 second overflow interrupts.
4      RTC.CLKSEL = RTC_CLKSEL_OSC32K_gc;
5
6      // Set the period to 32768, which corresponds
7      //to 1 second at 32.768 kHz.
8      RTC.PER = 32768;
9
10     // Enable overflow interrupt.
11     RTC.INTCTRL = RTC_OVF_bm;
12
13     // Enable the RTC.
14     RTC.CTRLA = RTC_RTCEN_bm;
15 }
```

9. Lab - Multi-Interrupt System

9.3 ADC Initialization

```c
static void ADC0_init(void)
{
    // Configure PD0 as input, disable digital
    // input buffer to save power
    PORTD.DIRCLR    = PIN0_bm;
    PORTD.PIN0CTRL  = PORT_ISC_INPUT_DISABLE_gc;

    // Set voltage reference for ADC to internal 2.048 V
    VREF.ADC0REF = VREF_REFSEL_2V048_gc;

    // Clock prescaler:
    // DIV20 -> 20 MHz / 20 = 1.000 MHz (within spec)
    ADC0.CTRLC = ADC_PRESC_DIV20_gc;

    // Select the positive input for ADC0 as AIN0 (PA0).
    ADC0.MUXPOS = ADC_MUXPOS_AIN0_gc;

    // Enable the ADC interrupt on result ready.
    ADC0.INTCTRL = ADC_RESRDY_bm;

    // Enable the ADC.
    ADC0.CTRLA = ADC_ENABLE_bm;
}
```

9. Lab - Multi-Interrupt System

9.4 RTC ISR

```
ISR(RTC_CNT_vect)
{
    // Clear the interrupt flag.
    RTC.INTFLAGS = RTC_OVF_bm;

    // Toggle the LED.
    LED_PORT.OUTTGL = LED_PIN;

    // Start an ADC conversion.
    ADC0.COMMAND = ADC_STCONV_bm;
}
```

9.5 ADC ISR

```
ISR(ADC0_RESRDY_vect)
{
    // Clear the interrupt flag.
    ADC0.INTFLAGS = ADC_RESRDY_bm;

    // Read the ADC value.
    adc_value = ADC0.RES;
}
```

9. Lab - Multi-Interrupt System

9.6 Complete Main Program

```c
#include <avr/io.h>
#include <avr/interrupt.h>

volatile uint16_t adc_value;

int main(void) {
    // Initialize LED
    LED_init();

    // Initialize RTC
    RTC_init();

    // Initialize ADC0
    ADC0_init();

    // Enable global interrupts
    sei();

    while (1)
    {
        /* System mostly idle */
    }
}
```

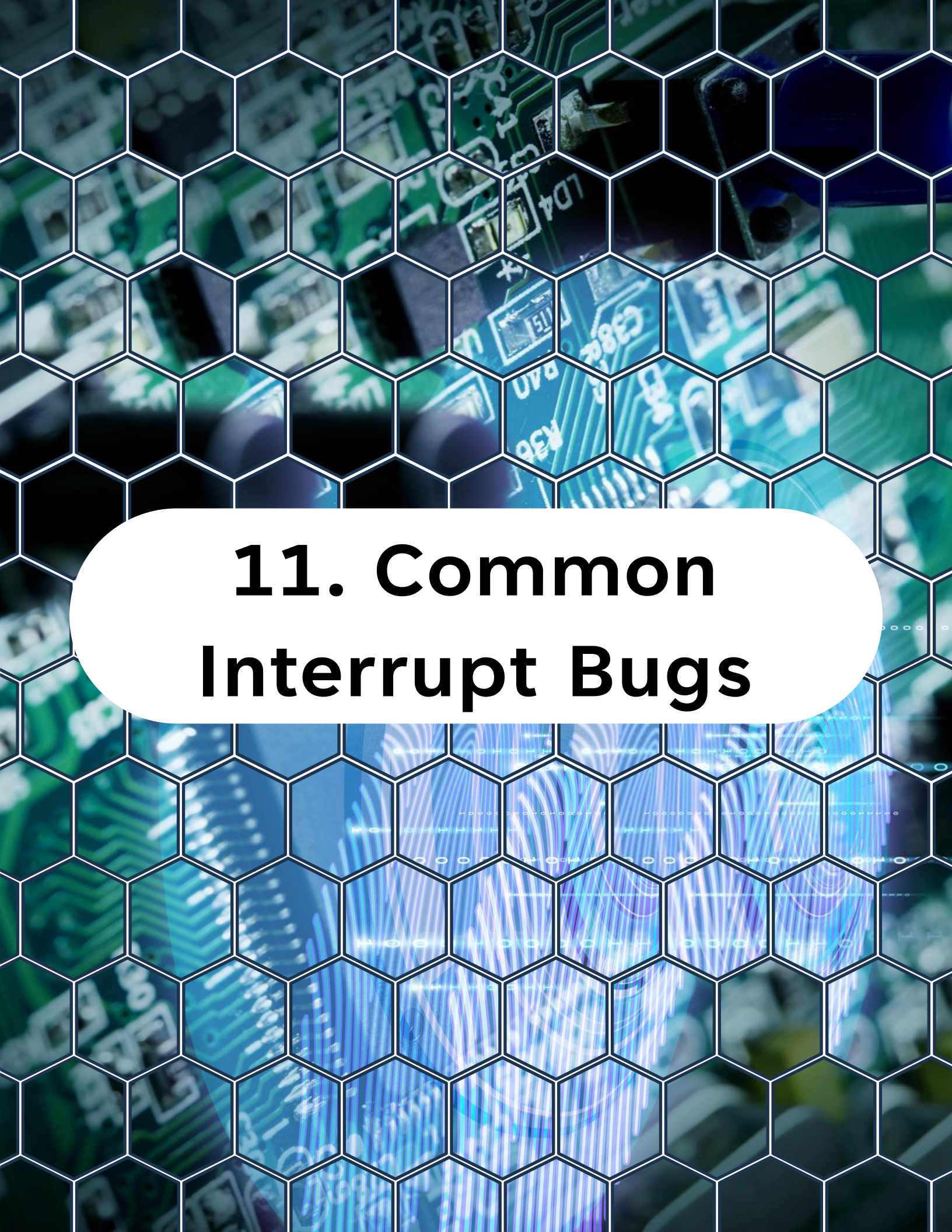
10. What You Should Observe

10. What You Should Observe

System behavior:

- RTC interrupt fires every second
- LED toggles every second
- ADC conversion every second
- ADC interrupt stores result

Multiple peripherals generate interrupts independently.



11. Common Interrupt Bugs

11. Common Interrupt Bugs

Typical mistakes include:

- Forgetting sei()
- Not clearing interrupt flags
- Missing volatile variables
- Writing long ISRs
- Calling blocking functions inside ISR

Always verify these first when debugging interrupts.

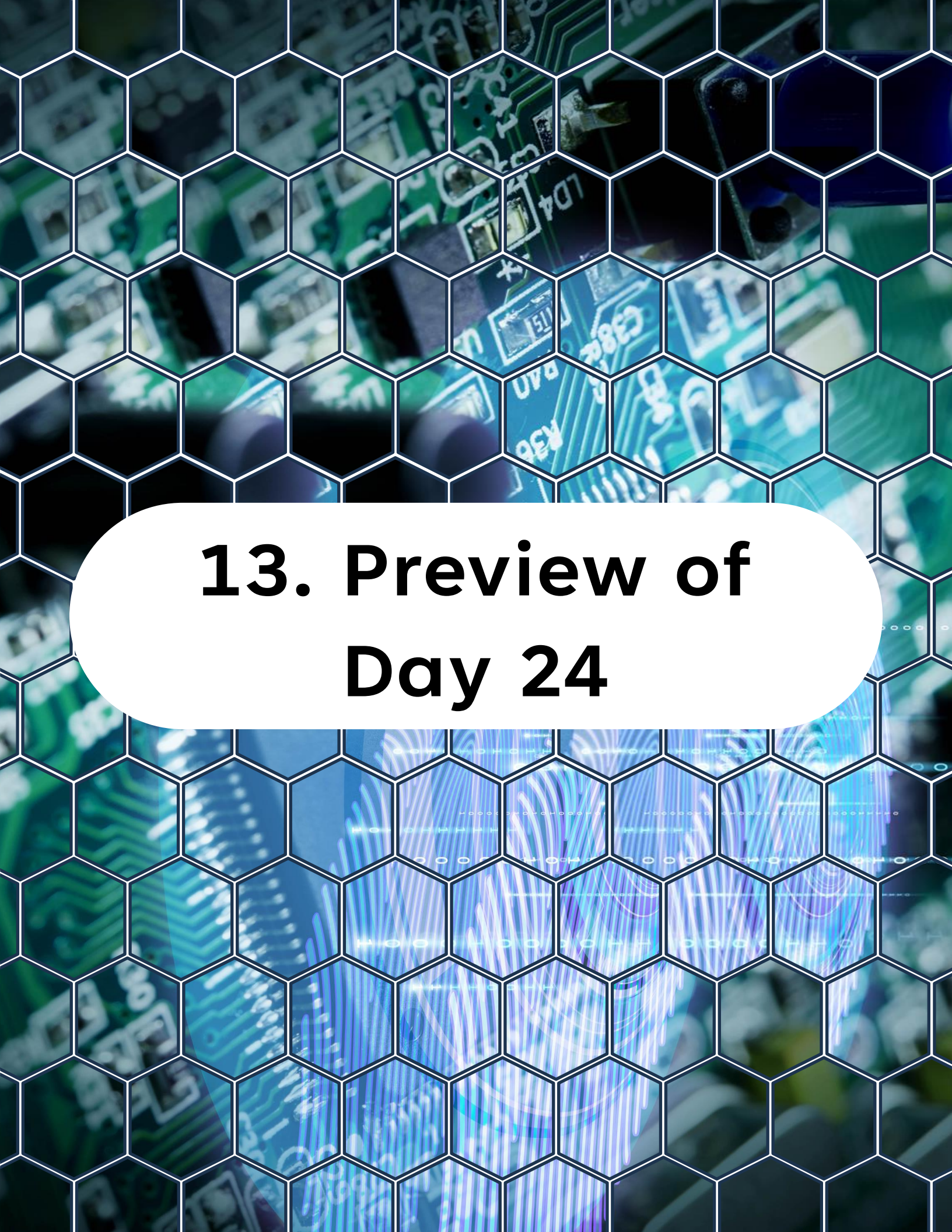
12. What You Should Understand Now

12. What You Should Understand Now

You now know:

- How interrupts work internally
- How vectors map to ISRs
- How to enable interrupts
- How to write safe ISRs
- How to handle multiple interrupt sources

Interrupts are the backbone of responsive embedded firmware.



13. Preview of Day 24

13. Preview of Day 24

Next:

Watchdog Timer (WDT)

You will learn how to build firmware that can recover from:

- Infinite loops
- Deadlocks
- Software crashes

This is essential for building reliable embedded systems.

You can find all source code, examples, and updates for this course in the GitHub repository, make sure to visit it, explore the files, and clone it so you can follow along hands-on.

https://github.com/god233012yamil/Bare-Metal-Embedded-Systems-with-AVR128DA48-Course